



Module: Développer en back-end

M205-complément1- Factories



Filière: Développement digital – option web full stack

Cours réalisé par:

- Asmae YOUALA (ISTA ADARISSA- Fès)

Introduction

- Laravel inclut la possibilité de remplir la base de données de manière beaucoup plus “propre” mais surtout beaucoup plus rapide et réutilisable grâce aux **Seeders** et aux **Factories**.
- Les **factories** permettent de créer des enregistrements en quantité et d'établir facilement diverses relations entre nos tables.
- les **factories** sont des **classes** qui étendent la classe **Factory** de base de Laravel et définissent une méthode **definition**.
- La méthode **definition** renvoie l'ensemble par défaut de valeurs d'attribut qui doivent être appliquées lors de la création d'un modèle à l'aide de la Factory.

Via l'assistant (helper) **fake**, les factories ont accès à la bibliothèque Faker PHP, qui vous permet de générer facilement différents types de données aléatoires à des fins de test et d'initiation.

Création

- Nous allons créer la classe Factory **StagiaireFactory** en exécutant la commande:

php artisan make:factory StagiaireFactory

- Les Factories créées se situent sous le dossier **/database/factories**

L'option **--model** de la commande permet de préciser de quel model s'agit-il.

php artisan make:factory StagiaireFactory --model=Stagiaire

```
class StagiaireFactory extends Factory
{
    /**
     * Define the model's default state.
     *
     * @return array<string, mixed>
     */
    public function definition(): array
    {
        return [
            'nom'=> fake()->firstName(40),
            'prenom'=>fake()->lastName(40),
            'age'=>fake()->numberBetween(18,30),
        ];
    }
}
```

Conventions et reconnaissance des modèles / Factory

- La méthode **HasFactory** du trait **factory** utilisera des conventions pour déterminer la fabrique appropriée pour le modèle auquel le trait est attribué. Plus précisément, la méthode recherchera une classe Factory dans l'espace de noms **Database\Factories** dont le nom de classe correspond au nom du modèle et est suffixé par **Factory**.
- Exemple** : Le modèle **Stagiaire** cherchera la classe Factory **StagiaireFactory** sous le répertoire **Database\Factories**
- Si ces conventions ne s'appliquent pas, on peut écraser la méthode **newFactory** de notre modèle pour renvoyer directement une instance de la classe **Factory** correspondante au modèle :

```
use Database\Factories\StgFactory;
use Illuminate\Database\Eloquent\Factories\Factory;

class Stagiaire extends Model
{
    use HasFactory;
    protected static function newFactory(): Factory
    {
        return StgFactory::new();
    }
}
```

- Ensuite, on définit une propriété **model** au niveau de la classe **Factory** correspondante:

```
protected $model = Stagiaire::class;
```

Utilisation

- On peut générer , par exemple, les données de dix stagiaires en écrivant:

```
Stagiaire::factory(10)->create();
```

- On peut aussi ajouter des données directement dans la méthode run() de la classe **DatabaseSeeder** sous le répertoire **database/seeder** :

```
public function run()  
{  
    \App\Models\Stagiaire::factory(10)->create([  
        'nom' => 'Alaoui',  
        'prenom' => 'Ahmed',  
        'age' => 18  
    ]);  
}
```

- La commande artisan pour remplir la base de données :

```
php artisan db:seed
```

Exemple:

- Soit la base de données du TP6: gestion des annonces
- Créons une classe Factory pour la table "annonces" via la commande: **php artisan make:factory AnnonceFactory**
- Ajoutons le code suivant à la classe créée:

```
public function definition(): array
{
    return [
        'titre' => $this->faker->words(3, true), // génère un titre avec 3 mots
        'description' => $this->faker->paragraphs(2, true), // génère un texte avec 2 paragraphes
        'type' => $this->faker->randomElement(['Appartement', 'Maison', 'Villa', 'Magasin', 'Terrain']),
        'ville' => $this->faker->city,
        'superficie' => $this->faker->numberBetween(50, 500), // génère un nombre entre 50 et 500
        'neuf' => $this->faker->boolean,
        'prix' => $this->faker->randomFloat(2, 10000, 1000000), // génère un prix entre 10,000 and
1,000,000 avec deux chiffres après virgule
        'photo' => $this->faker->imageUrl(640, 480, 'annonce', true), // génère un URL pour une fausse
image avec un texte basé sur le mot 'annonce'
        'user_id' => \App\Models\User::factory(), // génère et associe un utilisateur à l'annonce
        'created_at' => now(),
        'updated_at' => now(),
    ];
}
```

Exemple:

- Au niveau de la classe **DatabaseSeeder** ajoutons l'instruction qui permettra de générer 10 annonces:

```
class DatabaseSeeder extends Seeder
{
    /**
     * Seed the application's database.
     */
    public function run(): void
    {
        Annonce::factory(10)->create();
    }
}
```

- Il reste qu'à exécuter ce code en tapant la commande: `php artisan db:seed` sous le terminal